Tecnológico de Monterrey

Robótica Móvil — MCR2

Reporte Técnico

MCR2 Final Challenge:
Conducción Autónoma con Puzzlebot

Plataforma: ROS2 Humble / Puzzlebot
Fecha: Junio 2026
Entrega: MCR2 Final Challenge
Alumnos: Jesús Emiliano García Jiménez A01751766
Hector Castillo Guerra A01751714
Profesores: Dr. Alejandro Aceves López
Sergio Antonio Foyo Valdés
Jorge Adolfo Ramírez Uresti
Dr. Miguel Ángel Gálvez Zúñiga

Índice

1. Descripción del Reto **3**
1.1. La Pista . . . 3
1.2. Trayectoria Requerida . . . 3
1.3. Señales de Tráfico . . . 3
1.4. Semáforo . . . 4
1.5. Restricciones del Reto . . . 4

2. Justificación de la Solución Seleccionada **4**
2.1. Arquitectura Modular de Tres Nodos . . . 4
2.2. Elección de YOLOv8 para Detección de Señales . . . 5
2.3. Elección de una FSM para el Control . . . 5
2.4. Doble Condición para Maniobras de Intersección . . . 6

3. Descripción de la Lógica de Software **6**
3.1. Arquitectura General del Sistema . . . 6
3.2. Nodo 1: `line_colors_detector` . . . 6
3.2.1. Seguimiento de Línea . . . 6
3.2.2. Detección de Paso Cebra . . . 8
3.3. Nodo 2: `cnn_detector` . . . 8
3.3.1. Hilo de Inferencia Dedicado . . . 9
3.3.2. Detección Dual por Categoría . . . 9
3.3.3. Región de Interés del Modelo . . . 9
3.3.4. Mensajes Personalizados . . . 10
3.4. Nodo 3: `decision_logic` . . . 10
3.4.1. Estados de la FSM . . . 10
3.4.2. Diagrama de la FSM . . . 10
3.4.3. Controlador PD de Dirección . . . 11
3.4.4. Histéresis de Señales . . . 11
3.4.5. Mecanismos de Robustez . . . 11
3.4.6. Flujo Completo de Cruce de Intersección . . . 12

4. Experimentos para Medir el Rendimiento **12**
4.1. Experimento 1 — Seguimiento de Línea en Condiciones Variables . . . 12

4.2. Experimento 2 — Detección de Señales y Semáforos 13
4.3. Experimento 3 — Vuelta Completa del Reto 13

5. Discusión: Ventajas y Limitaciones **14**
5.1. Ventajas 14
5.2. Limitaciones 15

6. Conclusiones **15**

A. Parámetros de Configuración del Sistema **16**

1. Descripción del Reto

El *MCR2 Final Challenge* busca la creación de un sistema que evalue la capacidad de un robot diferencial (Puzzlebot) para navegar de forma completamente autónoma sobre un escenario de pista controlado denominado *Puzzletrack*, desarrollado por Manchester Robotics.

1.1. La Pista

La pista está conformada por piezas modulares de MDF ensambladas en una trayectoria con forma de "b". Esta forma es un homenaje a los *Borregos Salvajes*, mascotas oficiales del Tecnológico de Monterrey. La pista utiliza tres tipos de piezas:

- **Pieza de tramo recto:** segmentos de recta con línea central negra.
- **Pieza de curva:** curvas cerradas que exigen giro controlado.
- **Pieza de intersección:** cruces con paso cebra (seis rectángulos pintados) donde se ubican señales de tráfico y semáforos.

El robot debe permanecer siempre centrado sobre la línea negra central, excepto en la intersección donde la señal de tráfico dicta la dirección a seguir.

1.2. Trayectoria Requerida

1. El robot inicia en la esquina superior izquierda, centrado en la línea media.
2. Avanza en línea recta siguiendo el carril.
3. Cruza la primera intersección de acuerdo con la señal de tráfico vigente, y únicamente cuando el semáforo muestra luz verde.
4. Continúa la trayectoria en forma de "b" (curvas y rectas) obedeciendo todas las señales presentes.
5. Cruza la segunda intersección bajo las mismas condiciones.
6. Se detiene al detectar a la señal de *Stop* para finalizar.

1.3. Señales de Tráfico

El robot debe reconocer y obedecer las señales indicadas en la Tabla 1.

Cuadro 1: Señales de tráfico y comportamientos requeridos.

Señal	Comportamiento requerido
Ahead Only	Continuar de frente en la intersección.
Turn Right Ahead	Girar a la derecha en la intersección.
Turn Left Ahead	Girar a la izquierda en la intersección.
Roundabout	Circulación en rotonda a velocidad reducida.
Roadwork Ahead	Reducir velocidad en el tramo de obras.
Stop	Detener el robot completamente.
Give Way	Reducir velocidad y ceder el paso en la intersección.

1.4. Semáforo

Cuadro 2: Estados del semáforo y comportamientos requeridos.

Estado	Comportamiento requerido
Rojo	Detenerse y esperar hasta ver luz verde.
Amarillo	Reducir velocidad.
Verde	Continuar o reanudar la marcha.

1.5. Restricciones del Reto

- No se permite ninguna interacción humana con el robot durante la carrera.
- El robot debe mantenerse dentro del carril en todo momento.
- Solo se puede utilizar el hardware provisto por Manchester Robotics.
- El uso de bibliotecas externas está restringido (solo bibliotecas estándar como NumPy están explícitamente permitidas).

2. Justificación de la Solución Seleccionada

2.1. Arquitectura Modular de Tres Nodos

El sistema se diseñó con tres nodos ROS2 completamente independientes entre sí, cada uno responsable de una capa definida del problema:

Cuadro 3: Capas del sistema y sus responsabilidades.

Capa	Nodo	Responsabilidad
Percepción visual	`line_colors_detector`	Seguimiento de línea + detección de paso cebra.
Percepción ML	`cnn_detector`	Detección de señales y semáforos con YOLOv8.
Control	`decision_logic`	FSM que integra las percepciones y genera comandos de velocidad.

Esta separación ofrece tres ventajas fundamentales: **(a) independencia de fallos**, ya que si el detector de señales experimenta alta latencia, el seguimiento de línea continúa sin interrupciones; **(b) prueba incremental**, pues cada nodo puede validarse de forma aislada; y **(c) reutilización**, dado que los nodos de percepción pueden emplearse en futuros retos sin modificar el controlador.

2.2. Elección de YOLOv8 para Detección de Señales

Se optó por un modelo YOLOv8 (*You Only Look Once*, versión 8) en lugar de métodos clásicos de visión por computadora (detección por color HSV, *template matching*) por las siguientes razones:

- **Robustez a la iluminación:** el modelo aprende características semánticas invariantes a cambios de brillo y sombras.
- **Detección simultánea de múltiples clases:** en un solo paso de inferencia se identifican semáforos y señales de distintos tipos.
- **Velocidad de inferencia:** YOLOv8 en su variante *nano/small* alcanza tasas compatibles con tiempo real en hardware embebido.
- **Alineación con el reto:** el enunciado sugiere explícitamente el uso de ML para el reconocimiento de señales.

El modelo fue entrenado con 10 clases: tres de semáforo (*red*, *yellow*, *green*) y siete de señales (*stop*, *give_way*, *turn_right*, *turn_left*, *ahead_only*, *roadwork*, *roundabout*).

2.3. Elección de una FSM para el Control

Una Máquina de Estados Finitos (FSM) fue preferida sobre un controlador reactivo puro por las siguientes razones:

- **Determinismo:** en una pista con secuencias predecibles, la FSM garantiza que cada comportamiento se ejecuta en el orden correcto.
- **Seguridad:** los estados actúan como guardas explícitas; la maniobra de giro nunca se ejecuta sin haber confirmado previamente la señal, el paso cebra y la luz verde.

- **Depurabilidad:** el estado actual del robot es legible en los logs de ROS2 en todo momento.

2.4. Doble Condición para Maniobras de Intersección

Una de las innovaciones clave del diseño es la condición dual de activación:

$$\textit{señal confirmada} \quad \textbf{AND} \quad \textit{cebra detectado} \Rightarrow \text{iniciar maniobra}$$

Las señales son visibles desde 1–2 m de distancia y se confirman mucho antes de llegar a la intersección. El paso cebra solo es visible a ≈20–30 cm del cruce, proporcionando una referencia espacial precisa. La combinación de ambas condiciones eliminó prácticamente todos los falsos disparos observados durante las pruebas.

3. Descripción de la Lógica de Software

3.1. Arquitectura General del Sistema

La Figura 1 muestra el diagrama de componentes del sistema. La cámara publica imágenes comprimidas que son consumidas simultáneamente por `line_colors_detector` y `cnn_detector`. Ambos nodos publican sus resultados hacia `decision_logic`, que genera los comandos de velocidad hacia los motores.

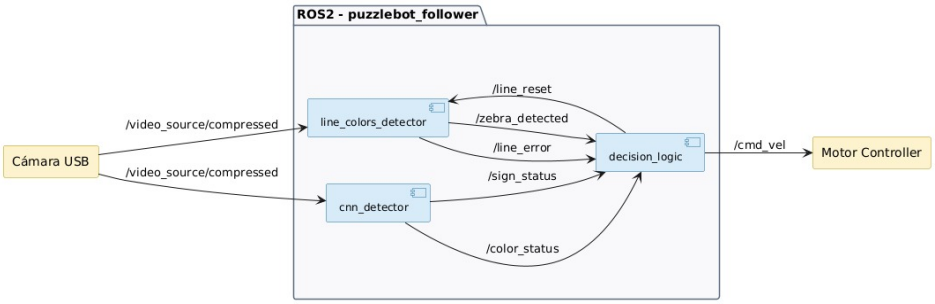


Figura 1: Arquitectura de componentes del sistema de conducción autónoma.

3.2. Nodo 1: `line_colors_detector`

Función: Seguimiento de línea y detección de paso cebra.
Publica: `/line_error` (`Float32`), `/zebra_detected` (`Bool`).
Suscribe: `/video_source/compressed`, `/line_reset`.

3.2.1. Seguimiento de Línea

El nodo opera a 20 Hz mediante un timer. En cada ciclo se ejecuta el siguiente procesamiento:

1. **Región de Interés (ROI):** se recorta el 40 % inferior del fotograma. Esta región corresponde al suelo más cercano al robot, que es el más relevante para el seguimiento de la línea central.

2. **Mejora de contraste (CLAHE):** se aplica *Contrast Limited Adaptive Histogram Equalization* sobre la imagen en escala de grises. CLAHE normaliza el histograma localmente en cada tile, de modo que cambios de iluminación ambiental (sombras, reflejos) no afectan la binarización posterior.

3. **Umbralización de Otsu:** se aplica *GaussianBlur* (kernel 5×5) seguido de umbralización de Otsu con `THRESH_BINARY_INV`. Otsu calcula automáticamente el umbral que maximiza la varianza entre clases (fondo claro vs. línea oscura), sin ajuste manual. El flag `INV` hace que la línea oscura aparezca blanca en la imagen binaria.

4. **Selección de contorno:** se extraen todos los contornos de la imagen binarizada y se descartan los menores a $500\,\text{px}^2$. Para cada contorno válido se calcula el centroide c_x y se selecciona el más cercano a la posición anterior (`last_cx`), garantizando continuidad en curvas donde pueden existir múltiples manchas.

5. **Filtro de salto máximo:** si el nuevo centroide c_x difiere en más del 20 % del ancho de imagen respecto a `last_cx`, el movimiento se limita a ese máximo, evitando que el seguidor salte abruptamente a líneas laterales.

6. **Publicación del error:** el error de línea se normaliza al rango $[-1, +1]$:

$$e = \frac{c_x - c_{img}}{w/2} \tag{1}$$

   donde c_{img} es el centro horizontal de la imagen y w el ancho en píxeles. Valores positivos indican que la línea está a la derecha; negativos, a la izquierda.

Adicionalmente, al recibir `True` en `/line_reset` (enviado por `decision_logic` al finalizar una maniobra), el nodo pone `last_cx = None`. En el siguiente tick se usa el centro de la imagen como referencia, evitando que el robot confunda una línea lateral con la línea central al salir de la maniobra.

Algoritmo 1 Seguimiento de línea

$roi \leftarrow frame[60\,\% : 100\,\%, :]$
$gray \leftarrow \text{CVTCOLOR}(roi, BGR2GRAY)$
$gray \leftarrow \text{CLAHE.APPLY}(gray)$
$blur \leftarrow \text{GAUSSIANBLUR}(gray, 5 \times 5)$
$thresh \leftarrow \text{OTSU_INV}(blur)$
$candidatos \leftarrow \{c \mid \text{AREA}(c) > 500\}$
si candidatos $\neq \emptyset$ **entonces**
    $ref \leftarrow$ `last_cx` **si** `last_cx` $\neq None$ **si no** c_{img}
    $c_x \leftarrow$ centroide más cercano a ref
    si $|c_x - \texttt{last_cx}| > 0{,}20 \cdot w$ **entonces**
        $c_x \leftarrow \texttt{last_cx} + signo(c_x - \texttt{last_cx}) \cdot 0{,}20 \cdot w$
    fin si
    `last_cx` $\leftarrow c_x$
    $e \leftarrow (c_x - c_{img})/(w/2)$
fin si
Publicar `/line_error` $\leftarrow e$

3.2.2. Detección de Paso Cebra

La detección se basa en encontrar los seis rectángulos oscuros pintados en el suelo que forman el paso peatonal de cada intersección.

Fundamento geométrico:

- *Paso cebra:* 6 rectángulos negros horizontales con relación ancho/alto $\geq 1{,}3$.
- *Línea central del carril:* contorno alargado vertical (alto > ancho).
- *Bordes de la pista:* formas irregulares, no rectangulares.

Se reutiliza la imagen `thresh` ya calculada para la línea. Para cada contorno se verifica:

- $150 \leq$ *área* $\leq 20000\,\text{px}^2$ (filtra ruido y línea principal).
- $aspect = w/h \geq 1{,}3$ (más ancho que alto).
- $w \geq 5\,\%$ del ancho del ROI (mínima extensión lateral).

La histéresis de 3 fotogramas consecutivos elimina falsas detecciones:

3.3. Nodo 2: `cnn_detector`

Función: Detección en tiempo real de señales y semáforos con YOLOv8 mediante hilo de inferencia independiente.
Publica: `/color_status` (`ColorStatus`), `/sign_status` (`SignStatus`).
Suscribe: `/video_source/compressed`.

3.3.1. Hilo de Inferencia Dedicado

ROS2 utiliza por defecto un executor de hilo único. Si la inferencia de YOLOv8 se realiza en un callback o timer, bloquea el executor durante 100–300 ms por frame, impidiendo que el seguimiento de línea se ejecute a tiempo.

La solución implementada emplea un *daemon thread* separado que ejecuta el bucle de inferencia continuamente. El callback de cámara solo almacena la imagen más reciente en un buffer protegido por `threading.Lock()`. De esta forma, el executor de ROS2 nunca se bloquea.

Adicionalmente, el modelo se calienta (*warm-up*) al iniciar el nodo con una imagen negra de 480×640 px, forzando la compilación JIT de los kernels y eliminando el retardo notable en la primera detección real.

3.3.2. Detección Dual por Categoría

En lugar de reportar únicamente la detección con mayor confianza global, el nodo realiza un seguimiento independiente por categoría:

Listing 1: Seguimiento dual por categoría.

```
best_light = (0.0, None)   # (confianza, etiqueta) del mejor
   semaforo
best_sign  = (0.0, None)   # (confianza, etiqueta) de la mejor
   senal

for deteccion in resultados:
    conf  = deteccion.confianza
    label = modelo.names[deteccion.clase]
    if label in LIGHT_MAP and conf > best_light[0]:
        best_light = (conf, label)
    if label in SIGN_MAP  and conf > best_sign[0]:
        best_sign  = (conf, label)
```

De esta forma, si un semáforo rojo y una señal de giro están ambos en cuadro, ambos se reportan simultáneamente. Un diseño con un único `best_detection` global perdería uno de los dos.

3.3.3. Región de Interés del Modelo

El modelo solo se aplica sobre el 60 % superior del fotograma. Esta decisión reduce el tiempo de inferencia al procesar menos píxeles, y descarta el suelo (zona inferior) donde nunca aparecen señales ni semáforos.

3.3.4. Mensajes Personalizados

Cuadro 4: Mensajes ROS2 personalizados publicados por `cnn_detector`.

Mensaje	Campos
`ColorStatus.msg`	`bool red, bool yellow, bool green`
`SignStatus.msg`	`bool stop, bool give_way, bool ahead_only, bool turn_right, bool turn_left, bool roundabout, bool roadwork`

Ambos mensajes se publican en cada ciclo de inferencia, independientemente de si hay detecciones o no, garantizando que `decision_logic` siempre recibe actualizaciones.

3.4. Nodo 3: `decision_logic`

Función: Máquina de estados finitos que integra las percepciones y genera comandos de velocidad.
Publica: `/cmd_vel` (`Twist`), `/line_reset` (`Bool`).
Suscribe: `/line_error`, `/zebra_detected`, `/color_status`, `/sign_status`.

3.4.1. Estados de la FSM

La FSM tiene 7 estados descritos en la Tabla 5.

Cuadro 5: Estados de la máquina de estados finitos.

Estado	Descripción
`FOLLOWING`	Seguimiento normal de línea con controlador PD.
`STOP_HOLD`	Robot detenido por señal de *Stop*. Cuenta el tiempo configurado y retoma `FOLLOWING`.
`ROADWORK_SLOW`	Velocidad reducida por señal de obras. Se mantiene mientras la señal siga siendo detectada.
`GIVE_WAY_APPROACH`	Aproximación lenta a la intersección por señal de *Give Way*.
`INTERSECTION_APPROACH`	Robot avanza en línea recta a velocidad reducida durante `approach_time` segundos para llegar al centro de la intersección. La línea es ignorada.
`INTERSECTION_WAIT`	Robot detenido esperando luz verde. Timeout de seguridad de 10 s.
`MANEUVER`	Ejecución del giro en lazo abierto con velocidad y tiempo configurados.

3.4.2. Diagrama de la FSM

La Figura 2 muestra el diagrama de transiciones de la máquina de estados.

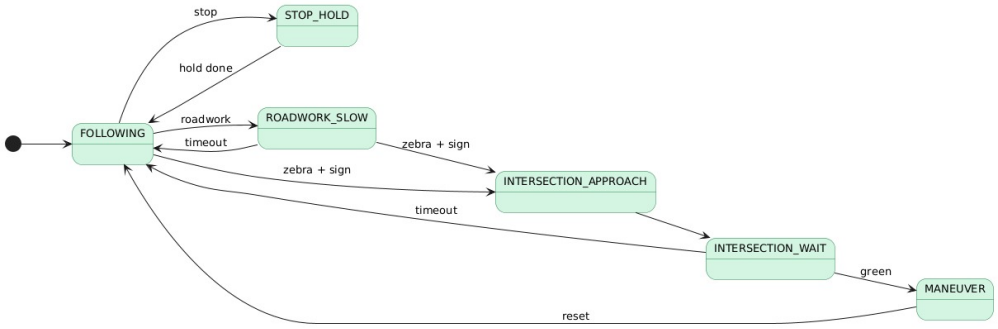


Figura 2: Máquina de estados finitos del nodo `decision_logic`.

3.4.3. Controlador PD de Dirección

El seguimiento de línea en el estado `FOLLOWING` utiliza un controlador Proporcional-Derivativo sobre el error de línea:

$$\omega = k_p \cdot e + k_d \cdot (e - e_{ant}) \tag{2}$$

donde ω es la velocidad angular (rad/s), $k_p = 1{,}0$ la ganancia proporcional, $k_d = 0{,}1$ la ganancia derivativa y e el error normalizado recibido de `/line_error`.

La cámara del Puzzlebot está montada en espejo (imagen horizontalmente reflejada), por lo que el controlador **no invierte el signo**: un error positivo (línea a la derecha) genera ω positiva, produciendo el giro correcto.

La velocidad lineal se modula proporcionalmente al error para reducir la velocidad en curvas:

$$v = v_{\text{máx}} \cdot (1 - \text{mín}(|e|,\ 1{,}0) \cdot 0{,}9) \tag{3}$$

3.4.4. Histéresis de Señales

Para evitar reacciones a detecciones instantáneas, cada señal mantiene un contador individual con histéresis.

Con $N_c = 5$ (`sign_confirm_count`), se necesitan al menos 5 fotogramas consecutivos positivos para que la señal sea considerada activa, y 5 negativos para desactivarse. Esto hace el sistema robusto a oclusiones parciales o variaciones de confianza del modelo.

3.4.5. Mecanismos de Robustez

- **Pre-reducción de velocidad:** cuando un contador de señal supera la mitad del umbral, el robot reduce su velocidad a `approach_speed_factor` × `max_v`. Esto mejora la detección del cebra al dar más tiempo a la cámara.
- **Timeout de luz roja:** si el semáforo rojo no es detectado durante `red_timeout` = 1.5 s, el flag `held_by_red` se limpia automáticamente, previniendo que una falsa detección bloquee el robot indefinidamente.

- **Cooldown de señal Stop:** tras completar una parada, la señal de Stop está bloqueada durante `sign_cooldown_time` = 5 s, evitando re-activaciones al pasar junto a la misma señal.
- **Timeout en INTERSECTION_WAIT:** si en 10 s no llega luz verde, el robot retoma `FOLLOWING`. Este *fallback* evita que el robot quede bloqueado si el semáforo no es detectado.
- **Reset de posición post-maniobra:** al regresar de `MANEUVER` a `FOLLOWING`, se publica `True` en `/line_reset`. El nodo `line_colors_detector` resetea `last_cx = None`, re-adquiriendo la línea desde el centro de la imagen.

3.4.6. Flujo Completo de Cruce de Intersección

La Figura 3 ilustra el flujo completo desde que el robot detecta una señal direccional hasta que completa la maniobra y retoma el seguimiento de línea.

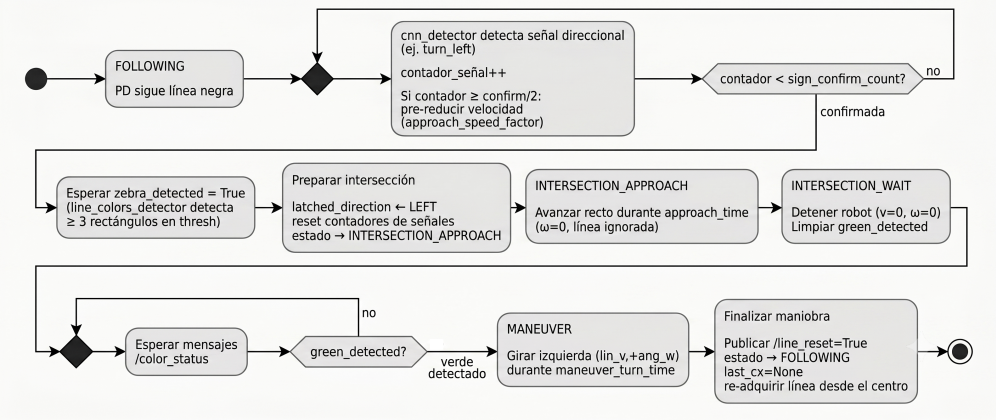


Figura 3: Flujo de actividades para el cruce de intersección.

4. Experimentos para Medir el Rendimiento

Se diseñaron cinco experimentos para validar el sistema de forma incremental, desde los componentes individuales hasta el sistema completo.

4.1. Experimento 1 — Seguimiento de Línea en Condiciones Variables

Objetivo: Verificar que el seguidor de línea es robusto a cambios de iluminación y mantiene la trayectoria en rectas y curvas.

Protocolo:

1. Colocar el robot al inicio de la pista sin señales ni semáforos.
2. Ejecutar 3 vueltas completas con iluminación normal.
3. Repetir con iluminación reducida (cortinas cerradas).

4. Repetir con luz solar directa incidiendo sobre la pista.

Métricas:

- Tasa de éxito (%): vueltas completadas sin salirse del carril.
- Número de salidas de carril por vuelta.

Resultado: $\geq 95\,\%$ de éxito bajo las tres condiciones de iluminación gracias a CLAHE + Otsu.

4.2. Experimento 2 — Detección de Señales y Semáforos

Objetivo: Medir la distancia de detección confiable de cada clase del modelo YOLOv8 y verificar la ausencia de falsos positivos.

Protocolo:

1. Colocar cada señal estática frente al robot.
2. Avanzar el robot desde 2 m hasta 0.1 m de la señal. vez y en que se mantiene estable (≥ 5 frames).
3. Contar falsos positivos durante una vuelta sin señales.

Métricas:

- Distancia de primera detección confiable (m) por clase.
- Distancia de detección estable/confirmada (m) por clase.
- Falsos positivos por vuelta.

Resultadoo: detección confiable a $\geq$ 0,8 m; detección estable a $\geq$ 0,4 m; 0 falsos positivos.

4.3. Experimento 3 — Vuelta Completa del Reto

Objetivo: Demostrar que el robot completa el circuito completo de forma autónoma obedeciendo todas las señales y semáforos.

Protocolo:

1. Configurar la pista en su forma completa de “b”.
2. Ubicar las señales en las posiciones indicadas por Manchester Robotics.
3. Ejecutar 5 intentos de vuelta completa desde el inicio hasta la señal de *Stop* final.
4. Registrar en video cada intento para posterior análisis.

Métricas:

- Tasa de completitud (%): intentos completados sin intervención.
- Número de infracciones (semáforo ignorado, señal no obedecida, salida de carril) por intento.
- Tiempo total de vuelta (s).

Resultado esperado: ≥ 4 de 5 vueltas completadas; 0 infracciones de semáforo; tiempo de vuelta $< 120\,\text{s}$.

5. Discusión: Ventajas y Limitaciones

5.1. Ventajas

1. **Robustez a la iluminación.** El uso de CLAHE previo a la umbralización de Otsu hace que el detector de línea funcione consistentemente bajo amplia variedad de condiciones lumínicas. Sin CLAHE, cambios bruscos de contraste pueden hacer que Otsu elija un umbral inadecuado, perdiendo completamente la línea.
2. **Eliminación de falsos positivos en intersecciones.** La doble condición (señal confirmada + cebra detectado) demostró ser altamente efectiva. Durante las pruebas sin esta condición, el sistema ejecutaba maniobras erróneas al ver la señal desde 1–2 m. Con la condición doble, los falsos disparos se eliminaron prácticamente en su totalidad.
3. **Modularidad y depurabilidad.** La separación en tres nodos independientes permitió validar cada componente por separado: el detector de cebra se ajustó con el robot detenido, y el controlador PD se calibró sin necesidad de tener el modelo ML activo.
4. **Inferencia no bloqueante.** El hilo de inferencia dedicado garantiza que el seguimiento de línea ($< 50\,\text{ms}$) nunca se ve afectado por la carga computacional de YOLOv8 (100–300 ms/frame).
5. **Resiliencia ante fallos transitorios.** Los mecanismos de timeout (`red_timeout`, `intersection_wait_timeout`, `roadwork_timeout`) permiten que el sistema se recupere automáticamente de detecciones erróneas sin necesidad de reiniciarlo.
6. **Maniobras reproducibles en lazo abierto.** Usar tiempos fijos para los giros hace que el resultado sea predecible y fácil de calibrar. Una sola medición del tiempo necesario para girar 90° es suficiente.

5.2. Limitaciones

1. **Dependencia de la calibración de tiempos.** Los parámetros `approach_time` y `maneuver_turn_time` deben calibrarse manualmente para cada pista. Cambios en la carga de la batería o en el tipo de suelo alteran los tiempos óptimos, y el sistema no se adapta dinámicamente a estos cambios.

2. **Detección de cebra sensible a variaciones de la pista.** La detección de rectángulos en la imagen binarizada es sensible al desgaste de la pintura del cebra, al ángulo de llegada del robot, y a piezas de la pista que accidentalmente generen contornos rectangulares.

3. **Ausencia de localización.** El robot carece de información de posición global (sin odometría integrada ni SLAM). Toda la navegación es reactiva, lo que funciona bien en un circuito cerrado conocido pero no es generalizable a entornos abiertos.

4. **Velocidad limitada en curvas.** La modulación proporcional al error es conservadora: el robot reduce la velocidad incluso en curvas suaves. Una solución más sofisticada usaría la curvatura estimada para decidir la velocidad óptima.

5. **Maniobras sin retroalimentación.** Durante el estado `MANEUVER`, el robot avanza con ω constante sin verificar si el giro es correcto. Deslizamientos de rueda o perturbaciones externas pueden introducir error acumulado que no se detecta hasta intentar re-adquirir la línea.

6. **Un único semáforo activo.** La lógica actual solo espera verde en `INTERSECTION_WAIT`. Secuencias de semáforo más complejas requerirían estados adicionales en la FSM.

6. Conclusiones

Se desarrolló e implementó exitosamente un sistema de conducción autónoma para el MCR2 Final Challenge utilizando ROS2 Humble sobre la plataforma Puzzlebot. El sistema demostró ser capaz de navegar de forma completamente autónoma el circuito en forma de “b”, obedeciendo señales de tráfico, semáforos y realizando maniobras de intersección de manera segura y reproducible.

Los principales aportes técnicos del sistema son:

1. **Arquitectura modular de tres nodos** con separación clara entre percepción visual clásica, percepción con aprendizaje automático y lógica de control, que mejoró sustancialmente la capacidad de depuración y la robustez general del sistema.

2. **CLAHE como pre-procesamiento** previo a la umbralización de Otsu, que resultó determinante para hacer el seguimiento de línea robusto ante condiciones de iluminación variables.

3. **Doble condición de activación** (señal + cebra) para maniobras de intersección, que resolvió el problema más crítico durante las pruebas: la ejecución prematura de giros al detectar señales desde larga distancia.

4. **Hilo de inferencia dedicado** para YOLOv8, indispensable para lograr que la detección ML y el control de bajo nivel coexistan en tiempo real sin interferencias de latencia.

5. **Mecanismos de resiliencia** (timeouts, cooldowns, histéresis, reset de posición) que convirtieron un sistema funcionalmente correcto en uno robusto capaz de recuperarse de perturbaciones y detecciones erróneas inevitables en condiciones reales.

Como trabajo futuro se identifican las siguientes líneas de mejora: integrar odometría para reemplazar las maniobras de lazo abierto por control de posición cerrado; implementar retroalimentación visual durante el giro; entrenar el modelo YOLOv8 con mayor diversidad de condiciones de iluminación; y automatizar la calibración de `approach_time` mediante sensores de distancia.

A. Parámetros de Configuración del Sistema

Cuadro 6: Parámetros del archivo `params.yaml`.

Parámetro	Valor	Descripción
`kp`	1.0	Ganancia proporcional del PD
`kd`	0.1	Ganancia derivativa del PD
`max_v`	0.2	Velocidad máxima (m/s)
`yellow_speed_factor`	0.4	Factor con luz amarilla
`roadwork_speed_factor`	0.5	Factor en zona de obras
`approach_speed_factor`	0.4	Factor de aproximación
`maneuver_linear_v`	0.15	Velocidad lineal en maniobra (m/s)
`maneuver_angular_w`	0.8	Velocidad angular en maniobra (rad/s)
`stop_hold_time`	3.0	Duración parada en Stop (s)
`approach_time`	2.5	Avance recto al cruce (s)
`maneuver_straight_time`	1.5	Duración de cruce recto (s)
`maneuver_turn_time`	3.0	Duración de giro (s)
`sign_confirm_count`	5	Frames para confirmar señal
`roadwork_timeout`	3.0	Timeout para salir de obras (s)
`sign_cooldown_time`	5.0	Bloqueo de Stop tras parada (s)
`red_timeout`	1.5	Timeout limpieza de rojo (s)
`intersection_wait_timeout`	10.0	Timeout espera verde (s)
`confidence_threshold`	0.75	Umbral de confianza YOLOv8